IB Comp Sci Notes

Kenny Lin / BusyPizza

October 8, 2026

Abstract

This guide / note is dedicated to the love of my life: computer science. Wish y'all have a great time studying this subject! (第一页用了纯黑背景其实是为了配合下面的插图，后面是正常的白底黑字)
Notes to Mr. Sanchez: this note is for my own use (and I might share this with classmates later this year once it reaches high level of completion), therefore you might see Chinese in it. Simply ignore it.



Contents

A Concepts of Computer Science **4**
A1 Computer fundamentals . . . 4
A1.1 Computer hardware and operation . . . 4
A1.1.1 Function and interaction of the main CPU components . . . 4
A1.1.2 Role of GPU . . . 5
A1.1.3 Difference between CPU and GPU . . . 5
A1.1.4 Purposes of different types of primary memory . . . 6
A1.1.5 FDE cycle . . . 7
A1.1.6 Pipelining in multi-core architectures . . . 9
A1.1.7 Internal and external types of secondary memory storage . . . 9
A1.1.8 Compression . . . 10
A1.1.9 Type of services in cloud implementation . . . 10
A1.2 Data representation and computer logic . . . 11
A1.2.1 Principal methods of representing data . . . 11
A1.2.2 How binary is used to store data . . . 12
A1.2.3 Purpose and use of logic gates . . . 16
A1.2.4 Constructing and analysing truth tables . . . 16
A1.2.5 Constructing logic diagrams . . . 20
A1.3 Operating systems and control systems . . . 20
A1.4 Translation . . . 20
A2 Networks . . . 20
A2.1 Network fundamentals . . . 20
A2.2 Network architecture . . . 20
A2.3 Data transmission . . . 20
A2.4 Network security . . . 20
A3 Databases . . . 20
A3.1 Database fundamentals . . . 20
A3.2 Database design . . . 20
A3.3 Database programming . . . 20
A3.4 Alternative databases and data warehouses . . . 20
A4 Machine learning . . . 20
A4.1 Machine learning fundamentals . . . 20
A4.2 Data preprocessing . . . 20
A4.3 Machine learning approaches . . . 20
A4.4 Ethical considerations . . . 20

B Computational Thinking and Problem Solving **21**
B1 Computational thinking . . . 21
B1.1 Approaches to computational thinking . . . 21
B2 Programming . . . 21
B2.1 Programming fundamentals . . . 21
B2.2 Programming constructs . . . 21
B2.3 Data structures . . . 21
B2.4 Programming algorithms . . . 21
B2.5 File processing . . . 21
B3 Object-oriented programming (OOP) . . . 21
B3.1 Fundamentals of OOP for a single class . . . 21
B3.2 Fundamentals of OOP for multiple classes . . . 21
B4 Abstract data types (ADTs) . . . 21

B4.1 Fundamentals of abstract data types 21

C Appendices **22**
C1 Introduction: Why do we live in a binary world? 22
C2 A binary / hex / decimal riddle: “The Palindrome Safe” 22
C2.1 Answer key 22

A Concepts of Computer Science

A1 Computer fundamentals

A1.1 Computer hardware and operation

A1.1.1 Function and interaction of the main CPU components

The CPU is the critical component that carries out the majority of the processing tasks in a computer system. Consists of:

- Control Unit (CU): Directs the operations of the processor. Responsible for the FDE cycle (managing all three operations) + directing the computer's memory, ALU & I/O devices to carry out the program instructions.
- Arithmetic Logic Unit (ALU): Performs arithmetic and logical operations.
- Registers: Small, fast storage locations within the CPU that hold data and instructions temporarily during processing.
  - Program Counter (PC): Holds the address of the next instruction to be executed.
  - Memory Address Register (MAR): Holds the address of the memory location that is currently being fetched.
  - Memory Data Register (MDR): Holds the data that is being transferred to or from memory.
  - Accumulator (ACC): Holds the result of arithmetic and logical operations performed by the ALU.
  - Instruction Register (IR): Holds the current instruction being executed.
- Buses: Communication pathways that transfer data between the CPU, memory, storage, and I/O devices.
  - Data Bus: Transfers data between the CPU, memory, and I/O devices. Bidirectional.
  - Address Bus: Transfers memory addresses from the CPU to memory. Unidirectional.
  - Control Bus: Transfers control signals from the CPU to other components. Bidirectional.

Core: the processing unit of the CPU. More cores generally means better performance, as multiple cores can handle multiple tasks simultaneously. However, the performance gain depends on the software's ability to utilize multiple cores effectively.
Single core processors work well for naturally sequential tasks, while multi-core processors excel in parallel processing tasks.
Cache: a small, fast memory located close to the CPU that stores frequently accessed data and instructions. It helps reduce the time it takes for the CPU to access data from main memory. (L1, L2, L3)
Co-processors: specialized processors designed to support the main CPU. Tasks from the CPU are offloaded to the co-processor so that they could run in parallel, which can perform them more efficiently. Examples include DSPs, GPUs, and AI accelerators.
Von Neumann vs Harvard architecture: Von Neumann architecture uses a single memory space for both instructions and data, while Harvard architecture uses separate memory spaces for instructions and data. Harvard architecture can provide better performance for certain applications, as it allows simultaneous access to both instructions and data. However, it can be more complex to implement and may require more hardware resources.

A1.1.2 Role of GPU

Used to accelerate the rendering of images, videos, and animations. It is particularly effective for tasks that can be parallelized, such as graphics processing and machine learning.

- GPUs have a large number of cores that can handle multiple tasks simultaneously, making them well-suited for tasks that require high computational power.
- They also handle the application of shaders and textures to 3D models, which includes lighting, shading and texture mapping.
- GPUs assist in decoding and encoding video streams, which is essential for video playback, video editing, and streaming applications.
- Due to the ability to perform parallel processing + matrix and vector operations, GPUs are widely used in machine learning and deep learning applications, where they can accelerate the training and inference of neural networks (esp. for deep learning models).
- GPUs also significantly outperformed CPUs in solving cryptographic puzzles, which is why they are widely used in cryptocurrency mining.

Accelerators: separate main circuit board with the processor, memory, power management, etc., but they can not operate by themselves. They are always part of a system (host) in which the CPUs run the operating systems and control the programs execution. (GPU can be one example as well)

A1.1.3 Difference between CPU and GPU

Heterogeneous programming: CPU acts as a main processor, controlling the execution workflow. The host makes all calls, allocates the memory, and handles the memory transfers between CPU and GPU. GPUs run tens of thousands of threads simultaneously on thousands of cores and does not do much of the data management. The device code is executed by doing calls to functions (kernels) written specifically to take advantage of the GPU.

Overview: The CPU is great for handling various jobs, but the GPU is better for doing the same job many times on a lot of data at once.
Usage scenarios:

- CPU: Used for general-purpose tasks that require flexibility, quick decisions, and coordination—such as running an operating system, web browsing, word processing, and managing program instructions.
- GPU: Used for tasks involving many similar calculations that can be carried out simultaneously / in parallel, such as rendering graphics in video games, training AI models, machine learning, video editing, and large simulations.

Processor	Processing	Architecture	Functionality
CPU	It is a **general** purpose processor, capable of handling many different tasks. It executes the instructions of computer programs, involving operations such as arithmetic, logic and controlling input / output (I/O) operations, as directed by the operating system.	CPUs generally have fewer cores. Each core is very versatile, making it capable of handling complex computations that require sequential processing.	Allows the user to switch between multiple tasks and applications. This makes it ideal for running the operating system and general software applications.
GPU	It is a **specialized** processor, with a focus on handling graphics, **rendering** images, video and animations.	Composed of hundreds or thousands of small cores that are well-suited for tasks that can be run in parallel. While each core is not as powerful as a standard CPU core, the high number of cores allows them to perform a large number of calculations simultaneously, making them perfect for graphical processing.	Suited for tasks that require simultaneous processing of large blocks of data, such as rendering images, video processing and deep learning applications.

In case of collaborative efforts to video games operation:

- The CPU handles the game' s core logic, including rules, physical calculations and AI behaviour. It processes the inputs from the player.
- The GPU renders the game' s visuals. It processes vertex and pixel data to draw images on to the screen, including 3D objects, textures and effects such as lighting and shadows.

A1.1.4 Purposes of different types of primary memory

It directly supports the execution of programs by providing the CPU with fast and flexible access to instructions and data. However, not all primary memory is equal: different types serve distinct purposes, trade-offs, and performance characteristics.

- RAM, Random Access Memory: holds instructions and data for programs that are currently running. Volatile (loses content when powered off) Directly accessible from CPU. Two types of RAM:
  - DRAM: capacitors + transistors (needs to be refreshed)
  - SRAM: flip flops to store data
- ROM, Read-only memory: used for storing instructions that are very rarely modified. (e.g. BIOS: initialize and test the system hardware components on startup, and to load the operating system (OS) software from the secondary memory storage into the RAM)
- Cache (Located within CPU): small, but provides high-speed access to the CPU compared to the RAM. It acts as a buffer between the CPU and the slower RAM, storing frequently used data and instructions. Caches are managed by the cache controller, which uses algorithms like Least Recently Used (LRU) to determine which data to keep or evict.

  - L1: located directly on the CPU, making it the fastest type of cache. However, it is also the smallest, often only a few kilobytes in size (32KB to 128KB per core). Each CPU core usually has its own L1 cache:
    * L1i to store instructions
    * L1d to store data
  - L2: either be on the CPU, or located very near the CPU. L2 cache is larger than L1 and can be up to several megabytes in size (256KB to 2MB per core), but slighly slower than L1 cache.
  - L3: located the furthest from the CPU chip. L3 cache may be shared on multiple-core CPUs. It is the largest of the three, and can be up to tens of megabytes in size (2MB to 64MB shared across all cores). It's also the slowest of all three.

- Registers: fastest and smallest type of memory in a computer system. Located directly inside the CPU, they provide temporary storage for data and instructions that the processor is actively using.
  - General-purpose: used to store operands, intermediate results, memory addresses, or anything else the executing program needs.
  - Special-purpose: used by the CPU control unit to manage and coordinate instruction execution.

Memory Hierarchy: Register ⇒ L1 ⇒ L2 ⇒ L3 ⇒ RAM ⇒ Secondary storage.

Cache hit: data being fetched is located within the cache, reduces retrieval time of data
Cache miss: data being fetched is only located within RAM / secondary storage.
Types of cache miss:

- Compulsory (cold): First-time access to data; it is unavoidable as it is inherent in the first reference to the data.
- Capacity: Cache too small to hold all data.
- Conflict: Multiple addresses map to the same cache line.

The percentage of hit rate determines the efficiency and effectiveness of the cache. Systems with a larger cache size will generally perform better, as well as systems with more intelligent prefetching techniques that can predict which data will be needed soon and load it into cache ahead of time.

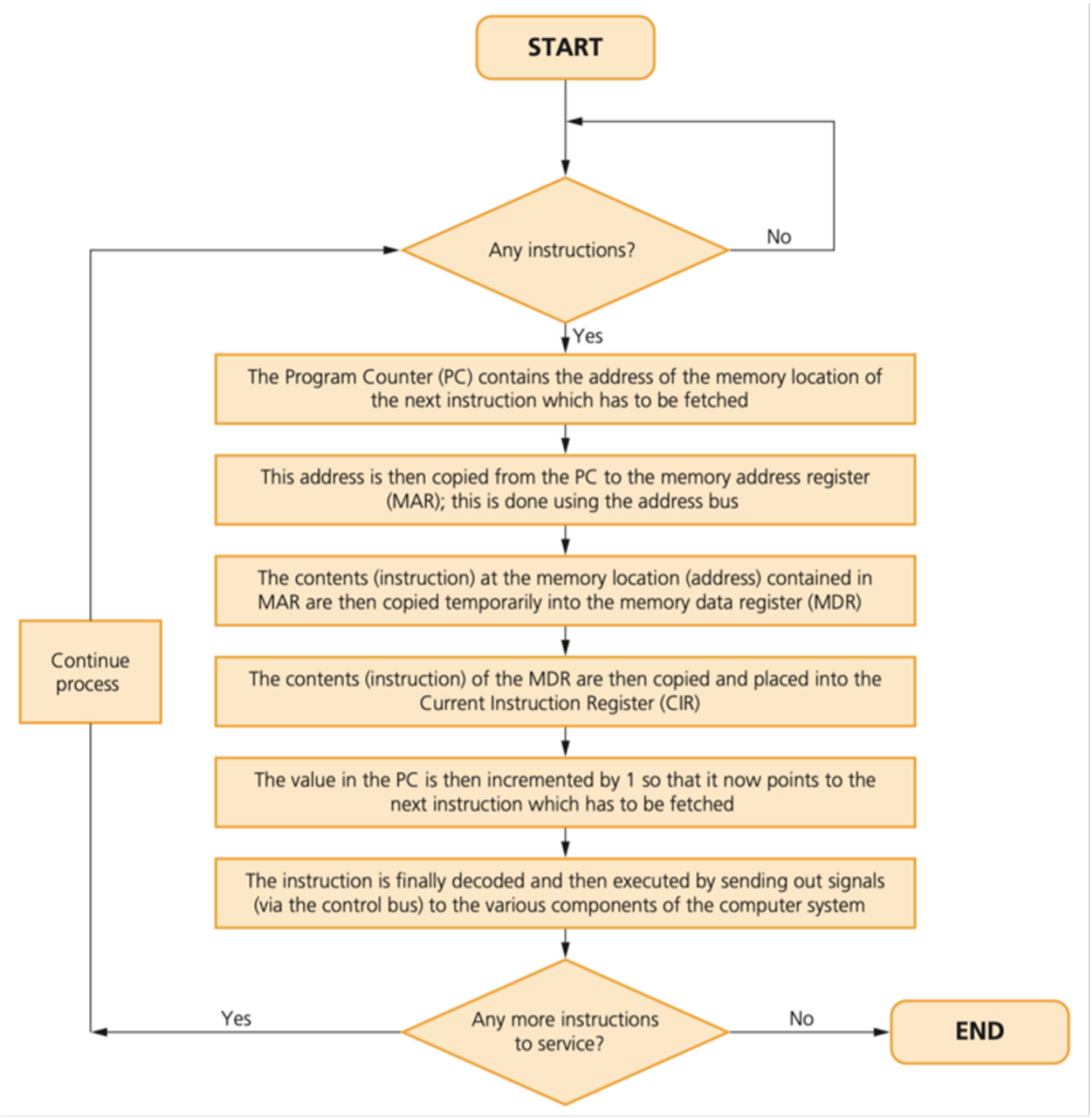
A1.1.5 FDE cycle

1. Fetch: The CPU fetches an instruction from the memory.
   - The Program Counter (PC) holds the address of the next instruction to be executed.
   - This address is copied into the Memory Address Register (MAR).
   - The address is sent to main memory through the address bus.
   - The instruction stored at that memory location is sent back to the CPU over the data bus and temporarily held in the Memory Data Register (MDR).
   - The instruction in the MDR is then transferred to the Current Instruction Register (CIR), where it awaits decoding.
   - Finally, the PC is incremented so it points to the address of the next instruction.
2. Decode: The CPU interprets the instruction and prepares the necessary operations to execute it.
   - The CU examines the opcode (the part of the instruction that specifies the operation) and determines what the CPU must do.

- If the instruction includes operands (addresses or data values), the CU prepares to retrieve them —possibly using the MAR and MDR again for memory access.
- Internal control signals are generated to direct data movement and select the appropriate ALU operation, register sources, or memory accesses.

3. Execute: The CPU performs the actions required by the instruction.
    - Arithmetic or logic operations: Data from registers are processed in the ALU, with results stored in the Accumulator (ACC) or another register.
    - Memory operations: The MAR and MDR are used again if data must be read from or written to RAM.
    - Control flow operations: The Program Counter (PC) may be updated.

说老实话，这段 IB 教科书上写的内容还不如 IG 时期的教科书，所以我就把去年的拿过来了：



Machine Language: lowest level of programming language that directly corresponds to the instructions executed by a computer's hardware. (In simple terms: binary.)

A1.1.6 Pipelining in multi-core architectures

A1.1.7 Internal and external types of secondary memory storage

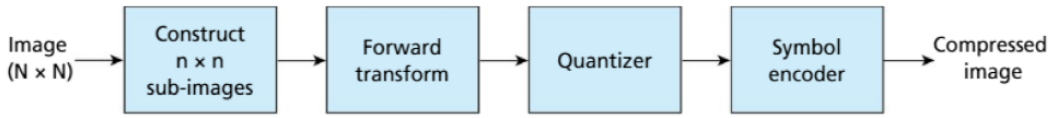
- Internal Storage
  - HDD: utilize a spinning magnetic disk to read / write data. They are suitable for storing large volumes of data, such as media files, backups, and documents, where speed is not so critical.
  - SSD: use flash memory to store data, offering high-speed data access and durability. This makes them very popular in portable devices such as laptops and tablets.
  - eMMC: a type of flash storage that utilizes NAND flash memory. They are soldered directly on to the motherboard of the device. While the capacity and speed do not match a standard SSD, their performance is adequate for basic computing needs and simple applications.
- External Storage
  - HDDs & SSDs: performance and comparison are identical to the internal versions.
  - Optical discs and optical drives (CD, DVD, blu-rays): cost of an optical disc is low compared to an HDD or SSD and, while their read / write speeds may be slower, they are sufficient for data archiving and playback. However, the discs are prone to scratches, especially if they are not stored correctly, and they require an optical drive to read and write to them.
  - Memory cards (SD, microSD, CF): compact storage devices often used in cameras, smartphones and other portable devices. They are ideal for expanding storage in mobile devices and for storing photos and videos in cameras, using NAND flash memory. Highly durable; read / write times are generally slower than SSDs, but outperform those of optical discs.
  - NAS: dedicated file storage connected to a network that allows multiple users to access data. It is often used in homes or businesses for centralizing data storage, file storage and data backup. Usually made up of multiple HDDs and SSDs configured in RAIDs. It is normally connected to the network via Ethernet, and runs a lightweight operating system designed for file storage, and the management and sharing of files.

Comparison between HDDs and SSDs:

Feature	**HDD (hard disk drive)**	**SSD (solid state drive)**
Storage technology	Magnetic storage with spinning disks and read / write heads	Flash memory with no moving parts
Speed	Slower read / write speeds (generally 50–150 MB/s)	Faster read / write speeds (generally 200–500 MB/s)
Durability	More prone to physical damage due to moving parts	More durable; resistant to physical shock
Noise	Produces noise due to moving parts	Silent operation
Power consumption	Higher power usage due to mechanical parts	Lower power consumption
Cost	Generally cheaper per GB	More expensive per GB
Capacity	Available in larger capacities (up to several TB)	Typically available in smaller capacities (up to several TB, but at a higher cost)
Weight	Heavier due to mechanical components	Lighter
Heat generation	Generates more heat due to moving parts	Generates less heat

A1.1.8 Compression

- Lossy: reduces files by permanently eliminating certain information. This information is redundant or less critical data, resulting in a compressed version that is not identical to the original but is, ideally, indistinguishable from the original to human senses.
  - Transform encoding:



    1. Transform coding takes an image of N × N size and sections it into smaller sub-images of size n × n.
    2. Then the forward transform is carried out on each of the sub-images. The forward transform can use different algorithms, depending on the type of file compression, but for JPEGs DCT (discrete cosine transform) is used. This takes the image data from the spatial domain (pixel values) to the frequency domain. The output breaks the sub-image down into low- and high-frequency coefficients.
       * Low-frequency data: correspond to slow changes in pixel values, such as broad areas.
       * High-frequency data: correspond to rapid changes in pixel values, representing fine details, edges and textures.
    3. These frequency coetficients are then passed to the quantizer. This step significantly reduces file size by simplifying the frequency coefficients obtained from the DCT. The purpose of quantization is to reduce the precision of high-frequency components (the fine details) rather than low-frequency components. This is because the human eye is less sensitive to high-frequency data loss compared to low-frequency detail. The extent of the quantization decides the compression level and the quality of the final image.
    4. The final step of transform coding is the symbol encoder. This is where the quantized coefficients are further compressed using entropy coding techniques. This runs through three further algorithms to reduce the file size by efficiently representing the frequency of occurrence of each symbol. The algorithms used at this stage are
       (a) Zigzag scan
       (b) RLE
       (c) Huffman coding

- Lossless: data is compressed to a smaller size, but can be restored back to the original without any loss of information. This technique works by identifying and eliminating statistical redundancy within the data, and this process can be reversed when needed.
  - Run-length encoding: used to reduce the size of files containing many consecutive repeated characters. The original string of info is represented by the recurrence of the letter + the letter itself. RLE is straightforward to implement and it is very effective for data with lots of repetitions, such as simple graphics and certain types of text file

A1.1.9 Type of services in cloud implementation

- SaaS: delivers software applications over the internet. SaaS provides a cost-effective and convenient solution for businesses and individuals, offering a wide range of applications from productivity tools to customer-relationship management systems.

- PaaS: provides a cloud-based platform that allows developers to build, test and deploy applications without managing the underlying infrastructure. PaaS includes tools & services to facilitate application development, such as databases, middleware (software that connects different applications) and dev frameworks. PaaS accelerates software development by allowing developers to focus on coding rather than infrastructure management.
- IaaS: provides virtualized computing resources over the internet, such as virtual machines, storage and networks. IaaS gives users full control over their virtual machines and networks. This reduces the need for upfront investment in hardware and allows businesses to rent solutions at a lower initial cost using a subscription model. IaaS is also scalable, making it easy to adjust resources as the user base grows.

A1.2 Data representation and computer logic

都是老生常谈的东西，看看就好。

A1.2.1 Principal methods of representing data

Modern computing ultimately settled on binary (base-2) as the primary number system. The base-2 system represents two possible states: 1 or 0. The binary system is particularly well-suited to represent the state of electrical switches within a computer system: on (1) and off (0). This simplicity reduces hardware complexity and enhances reliability.

The increased reliability of binary systems stems from their use of only two states. Small variations in signal strength do not affect data integrity as much as in systems using larger bases, making binary more robust in noisy (electrical disturbances) environments.

Conversion	Method
Binary → Decimal	Multiply each binary digit by its corresponding power of 2, then add the results. $1011_2 = 1(2^3) + 0(2^2) + 1(2^1) + 1(2^0)$ $= 8 + 0 + 2 + 1 = 11_{10}$
Decimal → Binary	Repeatedly divide the decimal number by 2, record the remainders, and read them from bottom to top. I'd personally recommend the subtraction method. (Subtract the decimal number by powers of 2 and record the results.) If you use this method, remember to start with the most significant bit (MSB) the left most bit, not the least significant bit (LSB) the right most bit.

Base-2 vs Base-10:

Kibibyte KiB	Mebibyte MiB	Gibibyte GiB	Tebibyte TiB	Pebibyte PiB	Exbibyte EiB	Zebibyte ZiB
1 KiB = 1024 bytes	1 MiB = 1024 KiB	1 GiB = 1024 MiB	1 TiB = 1024 GiB	1 PiB = 1024 TiB	1EiB = 1024 PiB	1 ZiB = 1024 EiB
Kilobyte KB	**Megabyte MB**	**Gigabyte GB**	**Terabyte TB**	**Petabyte PB**	**Exabyte EB**	**Zettabyte ZB**
1 KB = 1000 bytes	1 MB = 1000 KB	1 GB = 1000 MB	1 TB = 1000 GB	1 PB = 1000 TB	1 EB = 1000 PB	1 ZB = 1000 EB

1 byte = 8 bits, 1 nibble = 4 bits. Note here that KB and Kb are different units.

Hexadecimal: base-16 number system, where 10 - 15 are represented by A to F.
Reasons for using hexadecimal:

- Ease of conversion between binary and hexadecimal
- Compact size of display (used in debugging tools, memory dumps and assembly language programming)

Conversion of hexadecimal ⇔ binary: split the binary in nibbles and turn into hexadecimal for assembling the result. The reverse also stands; i.e. convert each hexadecimal character to binary nibbles, and assemble them in the correct order. (For decimal conversion, I recommend conversion to binary first and then convert to hexadecimal)

A1.2.2 How binary is used to store data

- Characters and strings
  - ASCII encoding: designed to provide a common standard for the interchange of text data. Initially started out as 7-bit encoding system (later 8 bits for more character support). Taking the 8-bit system as an example:
    - 010..... represents uppercase letters, while 011..... represents lowercase letters.
    - The last 5 digits, when converted from binary to decimal, corresponds to the alphabet letters (e.g. 1 → a, 2 → b)
  - Unicode encoding: provides a unique number for every character, regardless of platform, program or language. Few different versions of Unicode:

	UTF-8	**UTF-16**	**UTF-32**
Variable length encoding	1–4 bytes per character	2 or 4 bytes per character	4 bytes per character
Note	Compatibility: backward compatible with ASCII	Surrogate pairs: for characters outside the **Basic Multilingual Plane** (the maost commonly used characters and symbals for almost all modern languages), two 16-bit code units are used	Simplicity: easier to process because each character is exactly 4 bytes
Usage	Most commonly used encoding on the web and in many applications	Often used in Windows and Java environments	Less common due to higher storage requirements

  - Using UTF-8 as an example: (essentially kept ASCII but added some other bytes in a clever way): 11110000 10011111 10011000 10000100
    - 11110000: 1111 means 4 bytes is needed to transmit the character, the 0 coming right after the 4 1s means the end of transmitting the number of bytes. The 3 0s afterwards are the data transmitted.
    - 10.....: 10 means the continuation bytes being sent, the numbers afterwards are transmitted data.
    - 0001 1111 0110 0000 0100 are the final data received.
  - UTF-8 has been adopted by the internet as the main character encoding system. Some problems:

    - Due to the variable length, some characterstake more space compared to single-byte encodings.
    - ... This can lead to larger file sizes in certain contexts.
    - The processing required to handle variable length encoding also requires more complex processing compared to fixed-length systems such as UTF-32.

- Images
  - Bitmap / raster images: reproduce images by using a grid of pixels, with each pixel assigned a specific colour and intensity. Some key concepts:
    - Pixel: short for "picture element", the smallest unit of a digital image / display, representing a single point in the image with a specific color and intensity.
    - Image resolution: the number of pixels contained within a digital image, typically expressed as the dimensions (width × height) in pixels, and sometimes as the pixel density (PPI/ DPI) for print quality.
    - Color / bit depth: the amount of bits used to describe the color
    - Formula for calculating image size:

      $$\text{image size (bits)} = \text{width} \times \text{height} \times \text{bit depth}$$

      However, in reality, this calculation is not completely accurate, as the image would require more data to store metadata and other header information.
    - Metadata: information that describes other data, providing context and details about the data's content, structure and attributes.
  - Ways to improve the image quality:
    - Increase image resolution. A higher resolution allows for greater detail and clarity. The trade-off for a higher resolution image is larger file size, which can impact storage and transfer efficiency. 依旧老生常谈
    - Increase color depth. When we increase the colour depth, it allows for a wider range of colours to be represented, resulting in more vibrant and accurate images. $2^{\text{bit depth}}$ = numbers of color available for representation. Mostly 24 bits for modern photos.
  - When working with graphics, RGB values are often shown in hexadecimal.

- Audio: (analog) a continuous signal that represents sound waves through variations of air pressure. These sound waves can be captured through input devices, such as microphones, which convert the sound waves into a digital signal, which is stored as binary. Process: (金波儿老生常谈)

  1. An analog to digital converter(ADC) samples the amplitude (loudness) of the sound at discrete intervals in a process known as sampling. The rate at which this happens is measured in Hertz (Hz) —the higher the Hertz, the more samples are recorded per second.
     - Amplitude: the magnitude of change in a sound wave, representing the loudness or intensity of the sound.
     - Sampling: the process of converting a continuous analogue signal into a series of discrete digital values by measuring the signal' s amplitude at regular intervals.
  2. This sample 1s then stored and represented as a numerical value in binary. The precision is determined by the bit depth. The larger the bit depth, the more possible values that can be used to describe the sample.
  3. Formula for calculating audio size:

     $$\text{file size (bits)} = \text{sampling rate} \times \text{bit depth} \times \#\text{ of channels}$$

4. Many formats to choose from for storing audio files: (They mainly differ by whether they are compressed or uncompressed)
   - Uncompressed formats store the raw binary data (e.g. WAV, AIFF)
   - Compressed formats use algorithms to reduce the file size for storage or transmission (e.g. MP3, FLAC)

- Video
  - Components include:
    - Frames (visual)
      - In their raw form, frames are stored the same as images, with each pixel having a value that can be represented using a colour model such as RGB.
      - To improve colour efficiency, frames are often converted from the RGB colour model to a different one such as YUV. This helps with compression, as this colour model emphasizes luminance (brightness), which the human eye is more sensitive to than changes in colour detail.
    - Audio tracks
    - Metadata
    - Subtitles and closed captions
  - Compression methods / techniques
    - Spatial compression: particularly effective and commonly used for video that has significant detail variation in each frame. It reduces file size by eliminating redundant information within each frame, such as colour depth or detail levels.
    - Temporal compression: particularly effective and commonly used for video that has consistent motion across frames. It reduces file size by eliminating redundant information between consecutive frames, capturing only the changes or movements from one frame to the next. As a predictive compression technique, it predicts frame content based on the preceding and sometimes following frames, only storing the differences.

Different binary methods for storing integers:

- Unsigned binary (covered in A1.2.1)
- Signed binary (negative + positive):
  - Two's complement: most significant bit represents sign. 0 for positive, 1 for negative. To convert a positive binary number to its negative counterpart in two' s complement, you first invert all the bits and then add 1 to the least significant bit.
  - One' s complement: To obtain the one' s complement of a positive number, you simply invert all the bits (change 0s to 1s and 1s to 0s). One's complement have TWO (positive and negative) zeros.
  - Sign-magnitude: most significant bit serves for sign; the remaining bits represent the magnitude of the number, like how unsigned binary numbers work.
- Binary-coded decimal: method of representing decimal numbers where each digit of the decimal number is encoded separately into its own binary form. BCD assigns a 4-bit binary code to each decimal digit (0-9).
- Gray code (reflected binary code): a binary system where two successive values are only allowed to differ by one bit.

Decimal	3-bit binary	3-bit Gray code
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

For example, from decimal 3 to 4, ordinary binary changes from 011 to 100 (three bits change), while Gray code changes from 010 to 110 (only one bit changes).

- Excess-N (biased representation): a fixed bias (N) is added to the actual value to form an encoded value, and you subtract this bias to decode it. This is used to make all signed integers appear as non-negative binary numbers to allow for easier comparisons and arithmetic operations. 我打算法题的时候有一次这么干过 XD
- Fixed-point representation：a method used to store real numbers (numbers with fractional parts) in binary by fixing the position of the binary point. In a fixed-point system, the binary point is placed at a predetermined position, either between certain bits or at a specific location in the binary sequence. This allows for a straightforward representation of fractional numbers, though with some trade-offs in terms of precision and range.
- Floating-point representation: a method used to represent real numbers that can have a very large range or fractional parts. It does this by storing numbers in a format that includes a sign, an exponent and a mantissa (or significand).

  1. Sign bit (one bit): The sign bit determines whether the number is positive or negative.
  2. Exponent (eight bits): This is used to scale the number by a power of two and is stored using the Excess-127 system in its "biased" form.
  3. Mantissa (23 bits): This represents the significant digits of the number. The mantissa does not store leading ones (in normalized form, $1.xxxxxx \times 2^n$).

  An example of FP representation (32-bit single precision): represent 13.25_{10}.

  1. Convert to binary and normalize:

  $$13.25_{10} = 1101.01_2 = 1.10101_2 \times 2^3.$$

  2. Sign bit: the number is positive, so the sign bit is `0`.
  3. Exponent: add the bias of 127 to the actual exponent, giving $3 + 127 = 130_{10} = 10000010_2$.
  4. Mantissa: omit the implicit leading 1 and store the fractional bits `10101`, padded with zeros to 23 bits: `10101000000000000000000`.

  The stored representation is:

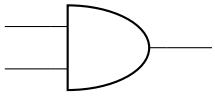
  $$\underbrace{\texttt{0}}_{\text{Sign (1 bit)}} \mid \underbrace{\texttt{10000010}}_{\text{Exponent (8 bits)}} \mid \underbrace{\texttt{10101000000000000000000}}_{\text{Mantissa (23 bits)}}.$$

  To decode it, subtract the exponent bias and restore the implicit leading 1:

  $$(-1)^0 \times (1 + 2^{-1} + 2^{-3} + 2^{-5}) \times 2^{130-127} = 1.65625 \times 8 = 13.25.$$

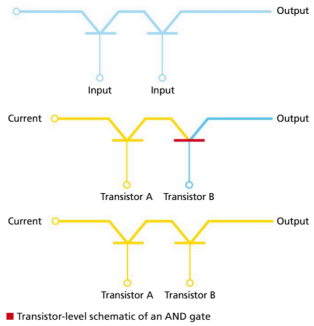
A1.2.3 Purpose and use of logic gates

Logic gates are made up of transistors, which act as electronic switches, allowing or blocking the flow of electrical current. In a transistor, the control wire (or "gate") regulates the current between the two electrodes, known as the "source" and the "drain". When voltage is applied to the gate, it allows current to flow from the source to the drain, enabling the transistor to switch states and perform logical operations.

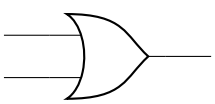


AND
$Y = A \cdot B$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

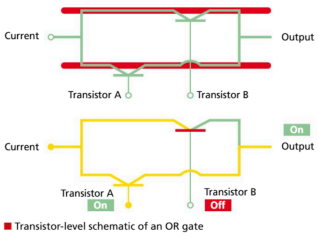


Transistor-level schematic of an AND gate

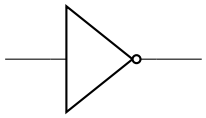


OR
$Y = A + B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

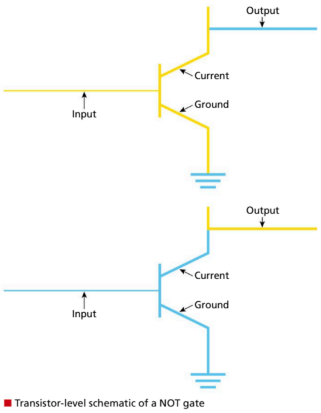


Transistor-level schematic of an OR gate

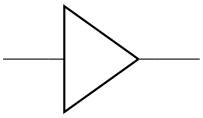


NOT
$Y = \overline{A}$

A	Y
0	1
1	0

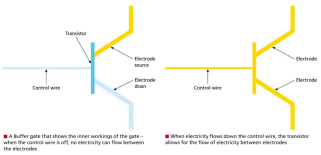


Transistor-level schematic of a NOT gate



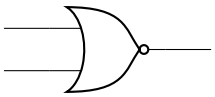
BUFFER
$Y = A$

A	Y
0	0
1	1



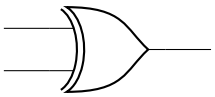
NAND
$Y = \overline{A \cdot B}$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



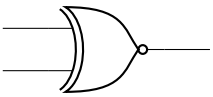
NOR
$Y = \overline{A + B}$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0



XOR
$Y = A \oplus B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0



XNOR
$Y = \overline{A \oplus B}$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

A1.2.4 Constructing and analysing truth tables

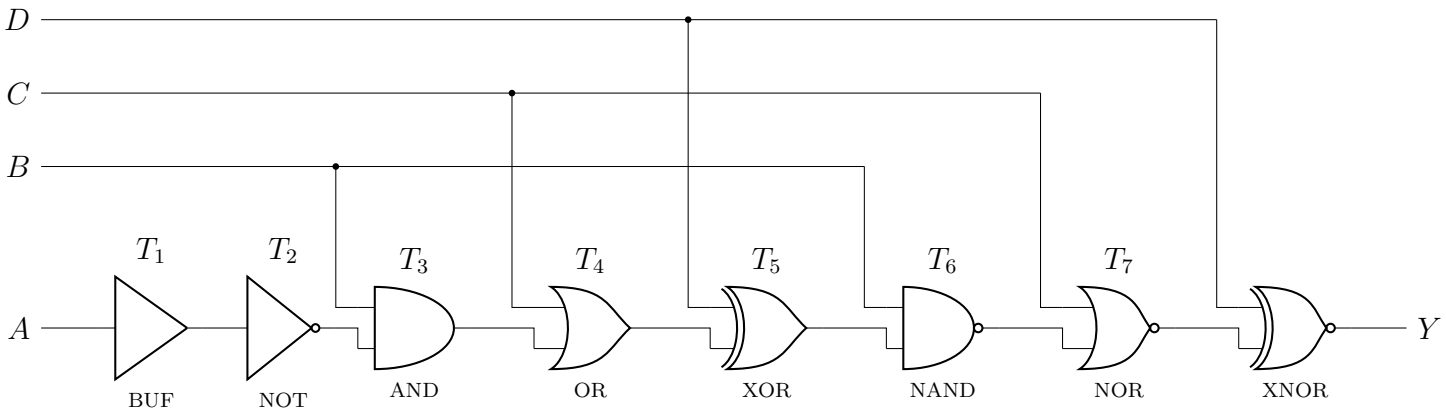
Starting with the intermediate values, work through the logic circuit.
Number of rows in truth table: $2^{\#\text{ of inputs}}$
Worked example: four inputs, all eight gate types.

Each gate output is labelled; every label becomes a column of the truth table.

$$
\begin{aligned}
T_1 &= \mathrm{BUF}(A) = A && \text{(BUFFER)}\\
T_2 &= \overline{T_1} = \overline{A} && \text{(NOT)}\\
T_3 &= T_2 \cdot B = \overline{A}\,B && \text{(AND)}\\
T_4 &= T_3 + C = \overline{A}\,B + C && \text{(OR)}\\
T_5 &= T_4 \oplus D = (\overline{A}\,B + C) \oplus D && \text{(XOR)}\\
T_6 &= \overline{T_5 \cdot B} && \text{(NAND)}\\
T_7 &= \overline{T_6 + C} && \text{(NOR)}\\
Y &= \overline{T_7 \oplus D} && \text{(XNOR)}
\end{aligned}
$$

Standard gate operators $X \uparrow Y = \overline{X \cdot Y}$ (NAND), $X \downarrow Y = \overline{X + Y}$ (NOR) and $X \odot Y = \overline{X \oplus Y}$ (XNOR),

$$Y = \Big(\big(((\overline{A} \cdot B) + C) \oplus D\big) \uparrow B\Big) \downarrow C \;\odot\; D.$$



Inputs				Intermediate values							Output
A	B	C	D	T_1 BUF	T_2 NOT	T_3 AND	T_4 OR	T_5 XOR	T_6 NAND	T_7 NOR	Y XNOR
0	0	0	0	0	1	0	0	0	1	0	1
0	0	0	1	0	1	0	0	1	1	0	0
0	0	1	0	0	1	0	1	1	1	0	1
0	0	1	1	0	1	0	1	0	1	0	0
0	1	0	0	0	1	1	1	1	0	1	0
0	1	0	1	0	1	1	1	0	1	0	0
0	1	1	0	0	1	1	1	1	0	0	1
0	1	1	1	0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0	0	1	0	1
1	0	0	1	1	0	0	0	1	1	0	0
1	0	1	0	1	0	0	1	1	1	0	1
1	0	1	1	1	0	0	1	0	1	0	0
1	1	0	0	1	0	0	0	0	1	0	1
1	1	0	1	1	0	0	0	1	0	1	1
1	1	1	0	1	0	0	1	1	0	0	1
1	1	1	1	1	0	0	1	0	1	0	0

Truth tables to determine outputs from inputs for a problem description: With a problem description, we first need to identify the inputs. Here we have three: alarm switch, baby crying and room temperature, which we can represent as A, B and C and set up our truth table.

Logical expressions: We can represent logic circuits using Boolean algebra. In Boolean algebra, parentheses are often used to indicate which operations should be performed first. If no parentheses are present, follow this sequence: NOT, AND, OR.

Karnaugh maps and algebraic simplification
Two input K-map: the four cells hold the output for the four possible combinations of A and B. For example, the function $F = A + B$ gives

A \ B	0	1
0	0	1
1	1	1

The two 1s in the column $B = 1$ group to give B, and the two 1s in the row $A = 1$ group to give A, so the K-map simplifies to $F = A + B$.

Three input K-map: the eight cells hold the output for the eight combinations of A, B and C, with the column labels in Gray-code order $00, 01, 11, 10$ so that adjacent cells differ by one variable. For example, the function $F = AB + C$ gives

C \ AB	00	01	11	10
0	0	0	1	0
1	1	1	1	1

The four 1s in the row $C = 1$ (that is, all four columns) group to give C, and the two 1s in the column $AB = 11$ (that is, $C = 0$ and $C = 1$) group to give AB, so the K-map simplifies to $F = AB + C$.

Another three input K-map: the function $F = \overline{A}BC + A\overline{B}C + ABC$ gives

C \ AB	00	01	11	10
0	0	0	0	0
1	0	1	1	1

The two 1s in the row $C = 1$ with $B = 1$ (that is, 01 and 11) group to give BC, and the two 1s in the row $C = 1$ with $A = 1$ (that is, 11 and 10) group to give AC, so the K-map simplifies to $F = BC + AC$.

Grouping conventions

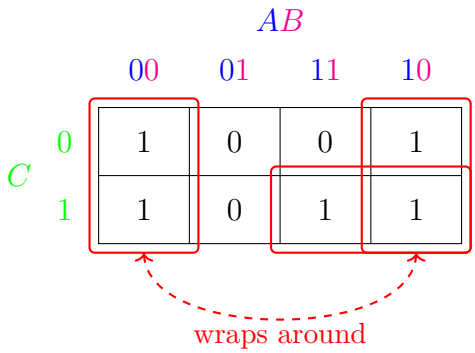
- Groups must contain powers of 2: One, two, four, eight or sixteen 1s can be grouped together.

- Groups must be rectangular or square: Each group should form a rectangle or square shape.
- Groups cannot be diagonal: Adjacent 1s can only be grouped horizontally or vertically, not diagonally.
- Groups must be as large as possible: Always aim to make the largest groups to simplify the expression further.
- Groups can overlap: Some 1s may be included in more than one group if it helps form larger groups.
- Minimize the total number of groups: The goal is to use the smallest number of groups to cover all the 1s.

To determine the expression from the groups, we look at each group of cells and refer to the variables. If a variable' s value stays the same across all the cells in the group, we keep that variable in the simplified expression. Iowever, if the variable' s value changes across the group, we discard it from the expression.

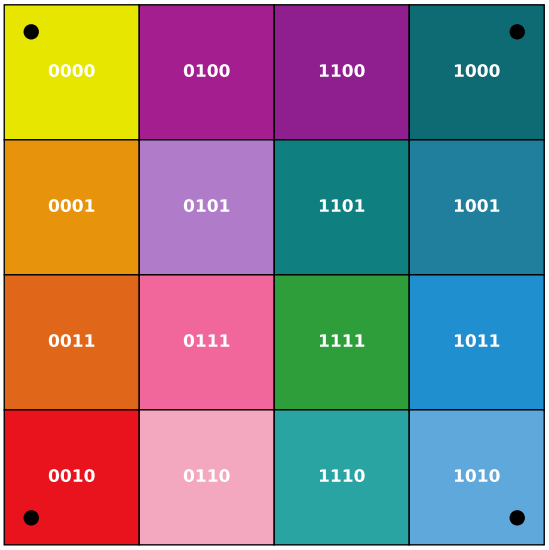
Wrapping around edges in K-maps

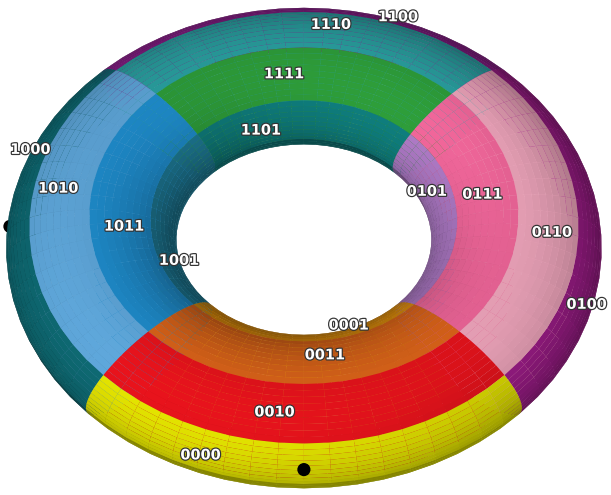
K-maps are considered three-dimensional, and groups can be formed from left to right and top to bottom (although only left to right is possible with three inputs). In the example of $F = \overline{B} + ABC$:



The four 1s in the columns $AB = 00$ and $AB = 10$ wrap around the left and right edges to form one group of four in which $B = 0$ throughout, giving $\overline{B}$; the remaining 1 groups with the adjacent 1 in $AB = 10$ (overlapping the $\overline{B}$ group) to give AC, so the K-map simplifies to $F = \overline{B} + AC$.

Four-variable K-maps wrap in both directions, so the map behaves like a torus: the left and right edges join, and the top and bottom edges join as well. The figure shows a 4×4 map and the same map drawn on a torus; the four wrapping corner cells are marked with dots.





A1.2.5 Constructing logic diagrams

By understanding the principles of Boolean algebra, we can simplify complex logic expressions and translate them into circuit diagrams. This journey from abstract mathematical notation to circuit design is essential for creating efficient and reliable digital systems. When a logic expression is given, follow the conventional operation sequence (parentheses first, then NOT, AND, OR...)

A1.3 Operating systems and control systems

A1.4 Translation

A2 Networks

A2.1 Network fundamentals

A2.2 Network architecture

A2.3 Data transmission

A2.4 Network security

A3 Databases

A3.1 Database fundamentals

A3.2 Database design

A3.3 Database programming

A3.4 Alternative databases and data warehouses

A4 Machine learning

A4.1 Machine learning fundamentals

A4.2 Data preprocessing

A4.3 Machine learning approaches

A4.4 Ethical considerations

B Computational Thinking and Problem Solving

B1 Computational thinking

B1.1 Approaches to computational thinking

B2 Programming

B2.1 Programming fundamentals

B2.2 Programming constructs

B2.3 Data structures

B2.4 Programming algorithms

B2.5 File processing

B3 Object-oriented programming (OOP)

B3.1 Fundamentals of OOP for a single class

B3.2 Fundamentals of OOP for multiple classes

B4 Abstract data types (ADTs)

B4.1 Fundamentals of abstract data types

C Appendices

C1 Introduction: Why do we live in a binary world?

At the lowest level, every device in a computer is built from transistors, and a transistor is essentially an electrically controlled switch: it is either conducting (on) or not conducting (off). Two states are trivially easy to represent, detect and store — a high or low voltage, a charged or uncharged capacitor, a magnetised or demagnetised spot. More importantly, the two states can be pushed far apart from one another, so small fluctuations caused by heat, interference or manufacturing tolerances will not flip a value by accident. A system with ten distinct voltage levels would have to pack them into the same range, leaving only a narrow margin between neighbours, and a slight drift would turn a "7" into an "8". Binary is therefore not just a convenient choice — it is the cheapest and most reliable thing we can physically build at scale.
The second reason is logical: two states map perfectly onto Boolean logic. TRUE/FALSE, 1/0, on/off — the logic gates (AND, OR, NOT, …) studied earlier are simply circuits that combine these two states. A machine whose hardware speaks Boolean is naturally addressed in base 2, so binary is the "native language" of the machine. Decimal is a human habit (we have ten fingers), and hexadecimal (base 16) is a human-friendly shorthand for binary: each hex digit is exactly four bits, so one byte is always exactly two hex digits. We do not live in a binary world because computers are simple; we live in it because two states are the simplest thing that can still be made reliable.

Dec	Bin	Hex	Dec	Bin	Hex
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

C2 A binary / hex / decimal riddle: "The Palindrome Safe"

A dusty safe sits in the corner of the CS lab, with a single dial that accepts a decimal number. Taped to the front is a note:

> *I am a number greater than 1. Written in binary, I read the same forwards as backwards. Written in decimal, I also read the same forwards as backwards. Written in hexadecimal, I fill exactly three digits. I am the* ***smallest*** *number with all three properties. What am I?*

C2.1 Answer key

1. "Exactly three hexadecimal digits" means the number lies between $100_{16} = 256_{10}$ and $FFF_{16} = 4095_{10}$. Since we want the smallest such number, we only need to search the decimal range from 256 upwards. (Smaller numbers such as 3, 5, 7, 9, 33 and 99 are already palindromes in both decimal and binary, but their hexadecimal forms have only one or two digits, so they fail the three-digit condition.)

2. Candidate decimal palindromes at or above 256: $262, 272, 282, 292, 303, 313, \ldots$

3. Convert each candidate to binary and test whether it is a palindrome:

   - $262_{10} = 100000110_2$ — not a palindrome.
   - $272_{10} = 100010000_2$ — not a palindrome.

- $282_{10} = 100011010_2$ — not a palindrome.
- $292_{10} = 100100100_2$ — not a palindrome.
- $303_{10} = 100101111_2$ — not a palindrome.

4. $313_{10} = 256 + 32 + 16 + 8 + 1 = 100111001_2$, which reads the same in both directions, and 313_{10} is itself a palindrome. Converting to hexadecimal confirms three digits: $313 \div 16 = 19$ remainder 9, and $19 \div 16 = 1$ remainder 3, giving $313_{10} = 139_{16}$ (i.e. `0x139`).

Answer: 313_{10}, which is 100111001_2 and 0x139.